



Solving problems

by searching

Lecturer: Dr. Nguyen Thanh Binh

Wednesday, 7:00-9:00





Outline

- *Problem-solving agents*
 - *Problem types*
 - *Problem formulation*
 - *Examples*
 - *Basic search algorithms*
- 

Problem-solving agents



function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns an action**

inputs: *percept*, a percept

static: *seq*, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null

problem, a problem formulation

state $\leftarrow$ UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then do**

goal $\leftarrow$ FORMULATE-GOAL(*state*)

problem $\leftarrow$ FORMULATE-PROBLEM(*state*, *goal*)

seq $\leftarrow$ SEARCH(*problem*)

action $\leftarrow$ FIRST(*seq*)

seq $\leftarrow$ REST(*seq*)

return *action*





Example

- *On holiday in Vietnam; at the moment in Nha Trang.*
 - *Flight leaves tomorrow in Hochiminh city.*
 - *Formulate goal: be in Hochiminh city.*
 - *Formulate problem:*
 - *States: Various provinces.*
 - *Actions: drive between provinces.*
 - *Find solutions: sequence of provinces.*
- 



Problem types

- *Deterministic, fully observable: single-state problem. Agents know exactly which state it will be in; solution is a sequence.*
- *No-observable: sensorless problem (conformant problem). Agent may have no idea where it is and solution is a sequence.*



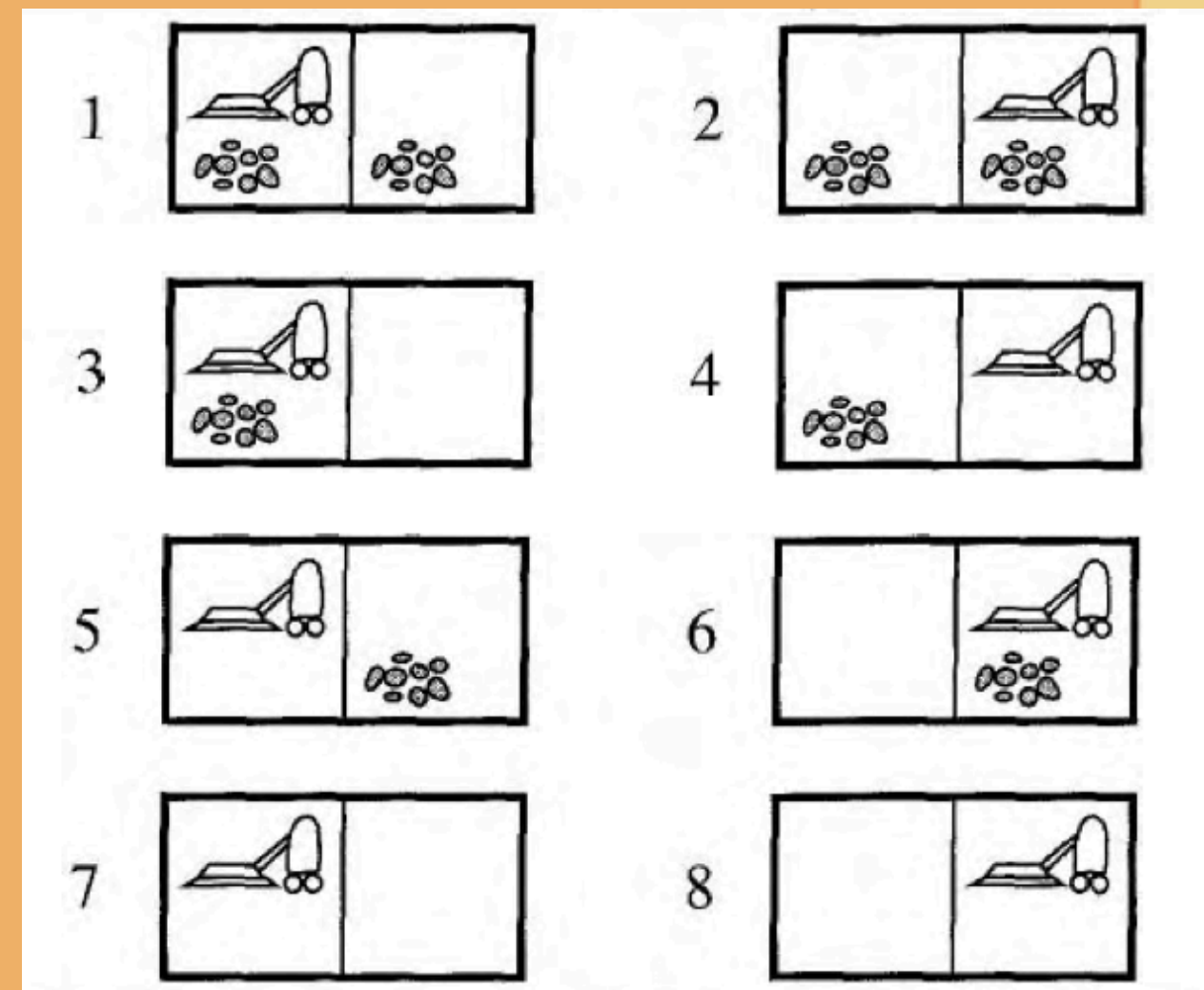
Problem types

- *Nondeterministic and/or partially observable:
contingency problem:*
 - *percepts provide new information about the current state*
 - *often interleave search, execution*
- *Unknown state space: exploration problem*

Example: vacuum world



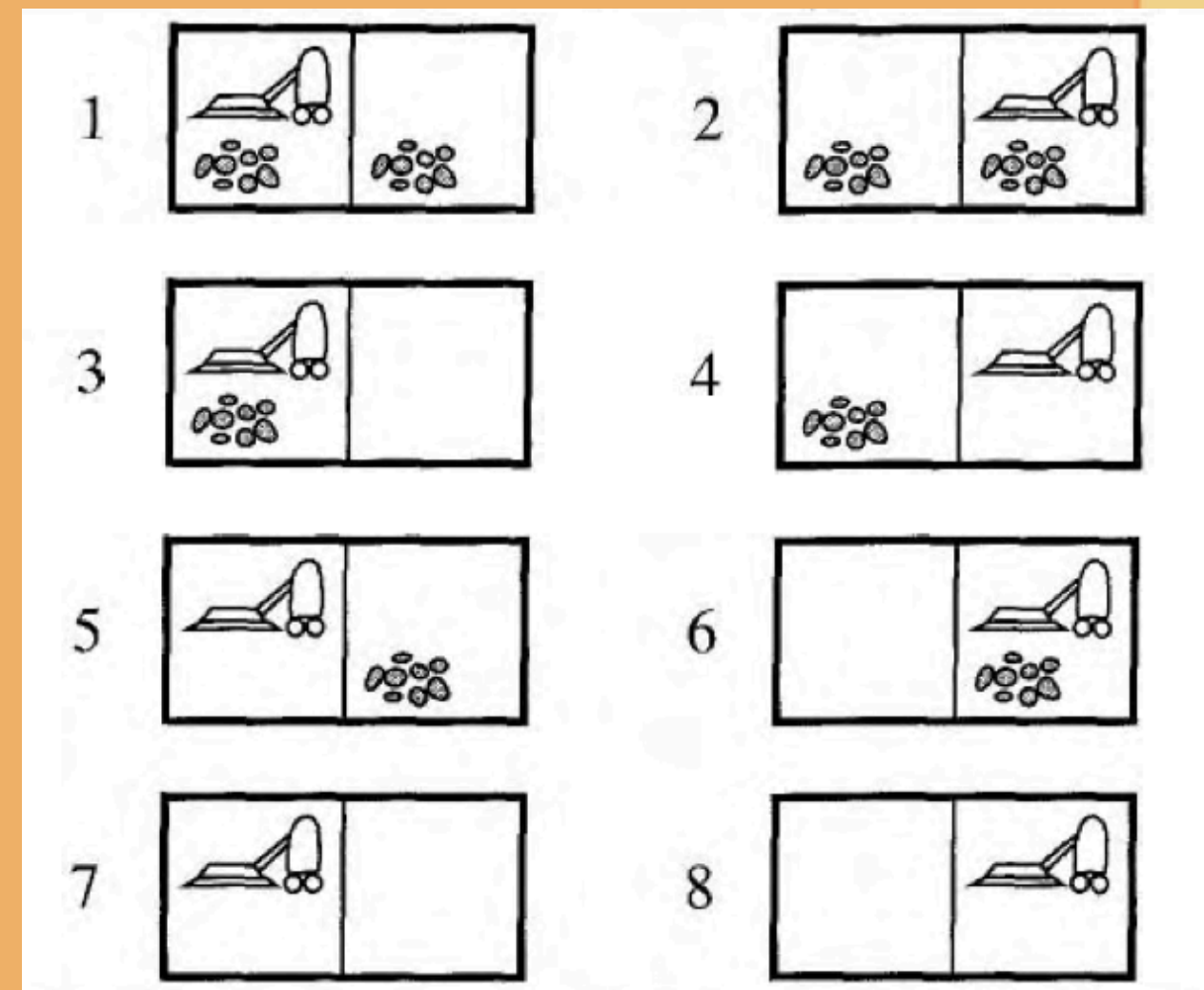
- Single-state, start in state 5.
- Solutions?



Example: vacuum world



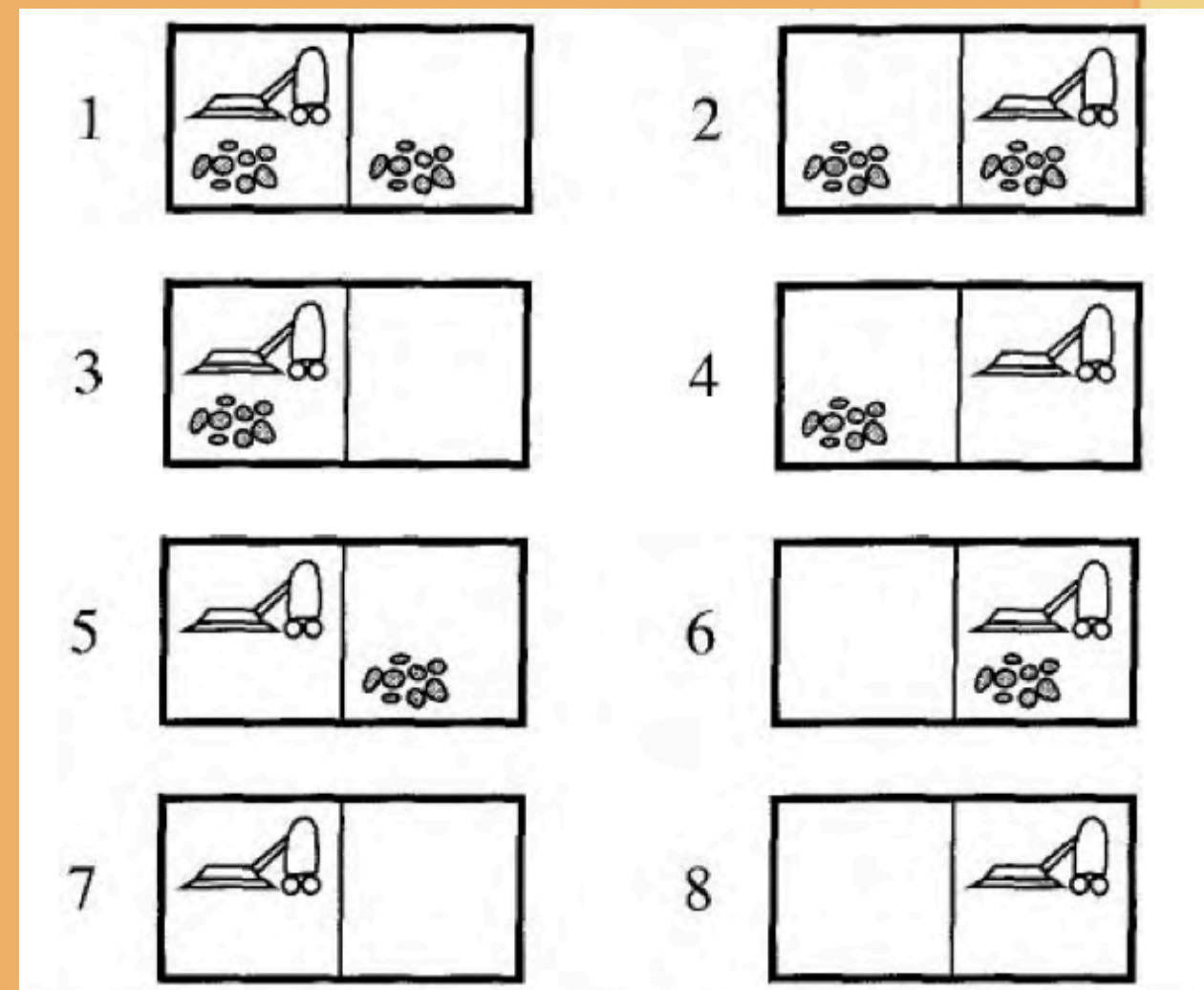
- Single-state, start in state 5.
- Solutions? [Right, Suck]



Example: vacuum world

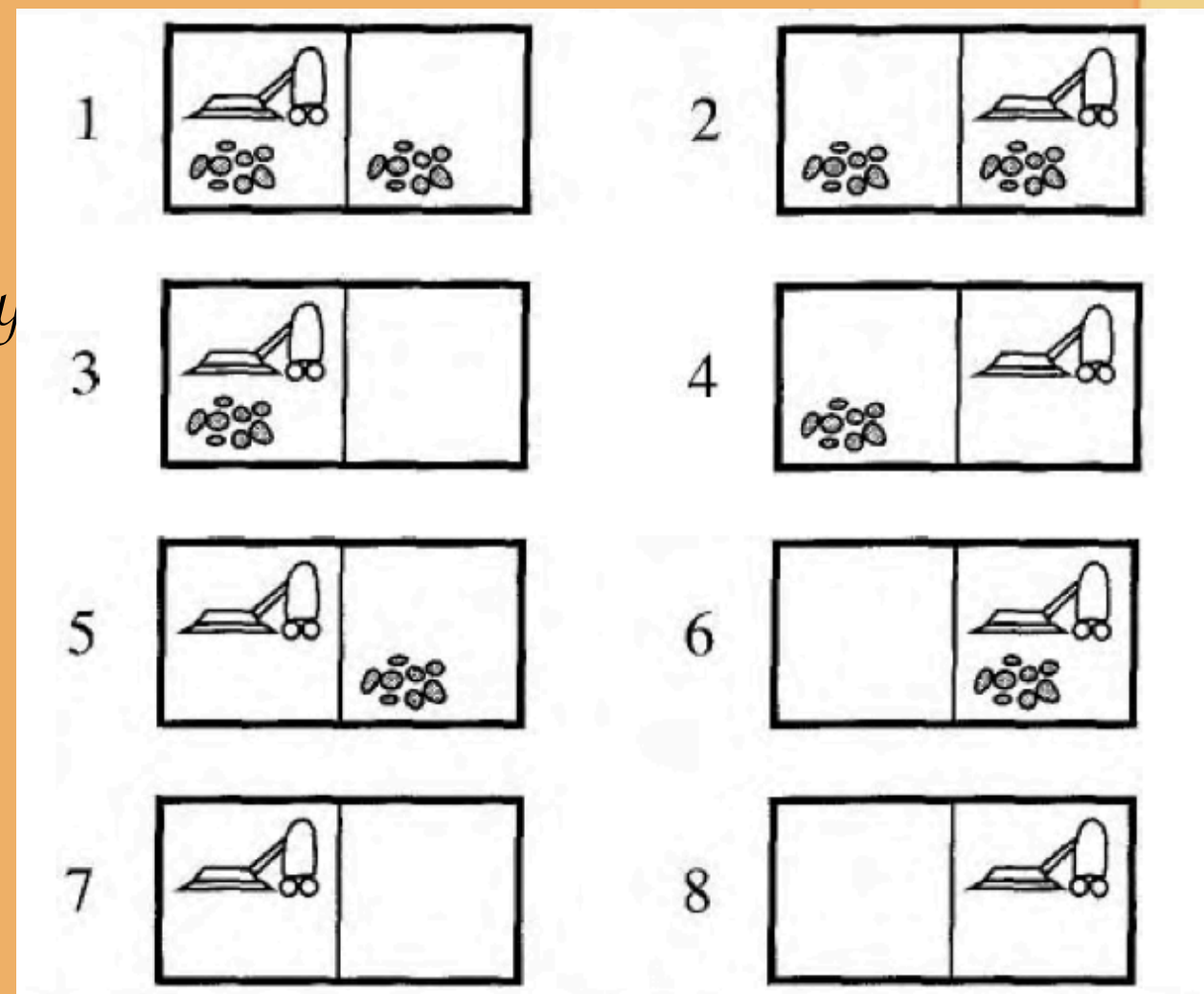


- Sensorless:
 - Start in $\{1, 2, \dots, 6, 7, 8\}$
 - Right goes to $\{2, 4, 6, 8\}$
- Solutions?



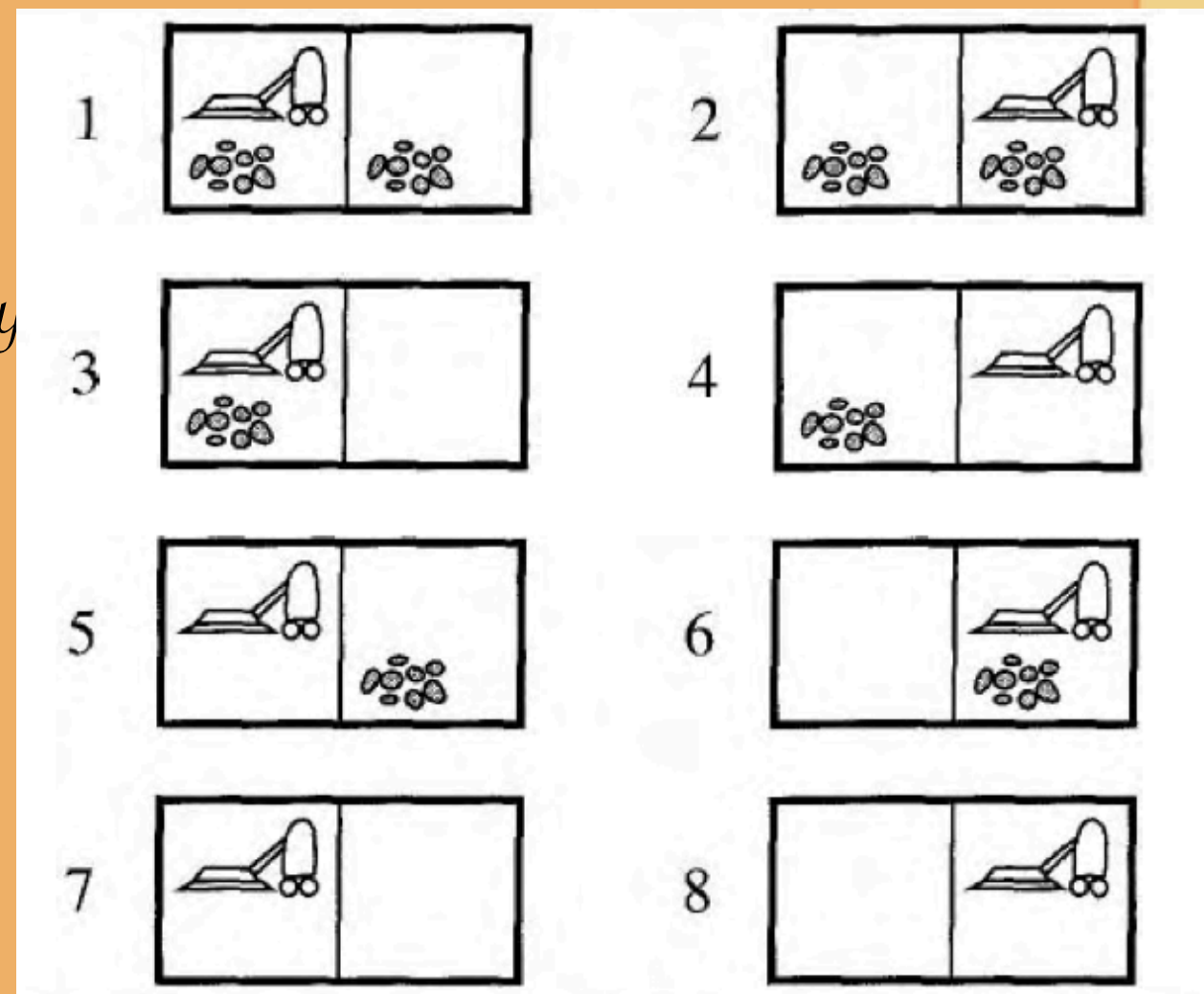
Example: vacuum world

- Contingency problems:
 - Nondeterministic: Suck may dirty a clean carpet.
 - Partially observable: location, dirty at current location.
 - Percept: $[L, \text{Clean}]$, i.e., start in the state 5 or 7.
 - Solutions?



Example: vacuum world

- Contingency problems:
 - Nondeterministic: Suck may dirty a clean carpet.
 - Partially observable: location, dirty at current location.
 - Percept: $[L, \text{Clean}]$, i.e., start in the state 5 or 7.
 - Solutions? [Right, if Dirty then Suck]



Single-state problem formulation



- One problem can be determined by the following items:
 - Initial state: where the agent starts in, e.g., “at Nha Trang”
 - Actions or successor function $S(x)$ = set of action-state pairs.
 - Goal test: explicit/implicit to determine whether a given state is a goal state.
 - Path cost: to show a numeric cost to each path, e.g., sum of distances, number of actions executed, ...
 - $C(x, a, y)$ = step cost (non-negative)



Single-state problem formulation



- *A solution of the problem is a sequence of necessary actions from the initial state to achieve the goal state.*
- *Solution quality is measured by the path cost function and an optimal solution has the lowest path cost among all possible solutions.*





Selecting a state space

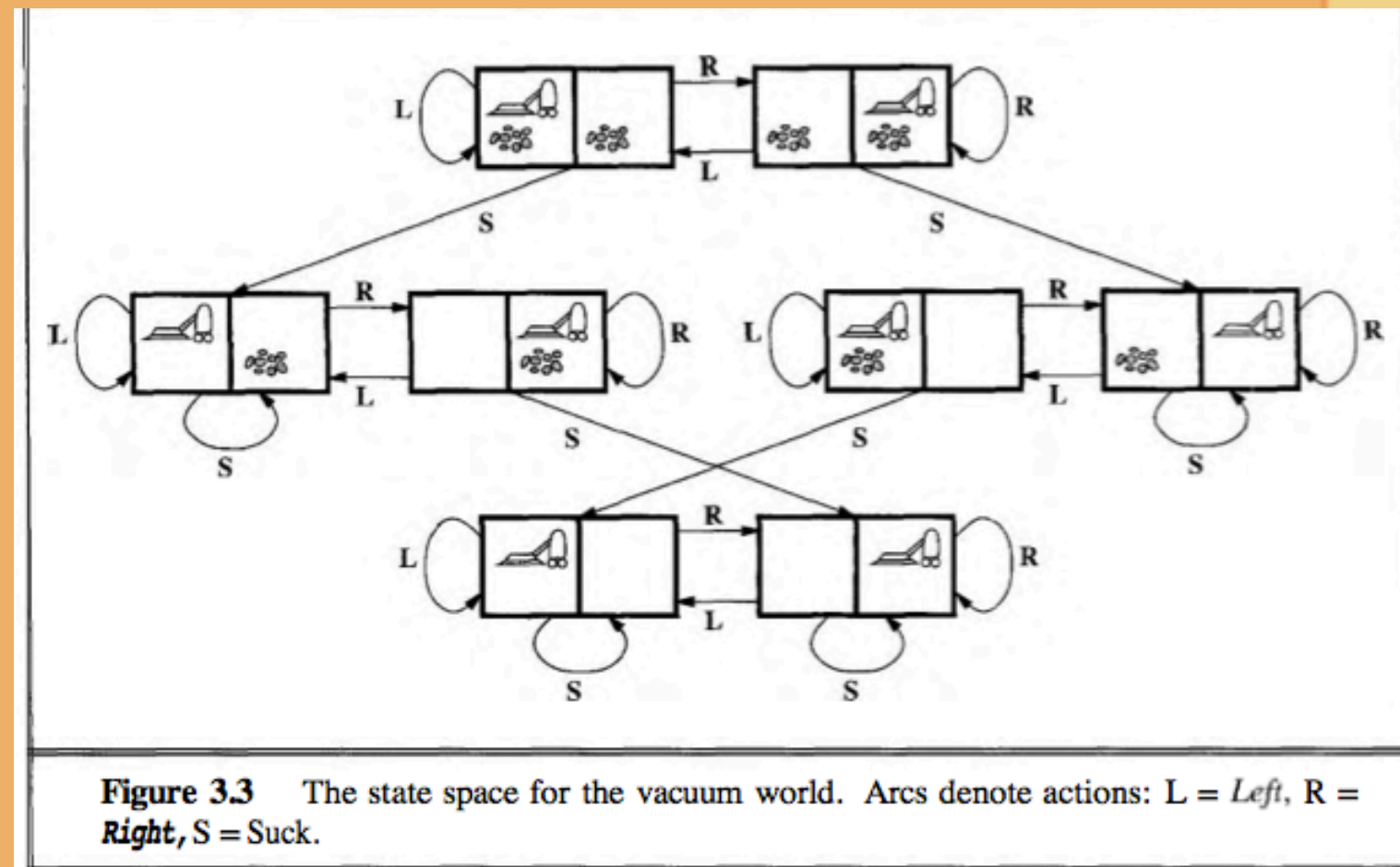
- *Real world is complicated. State space must be abstracted for problem solving.*
- *Abstract state = a set of real states.*
- *Abstract action = a complex combination of real actions.*
- *Abstract solution = a set of real paths that are solutions in the real world.*



Vacuum world state space graph



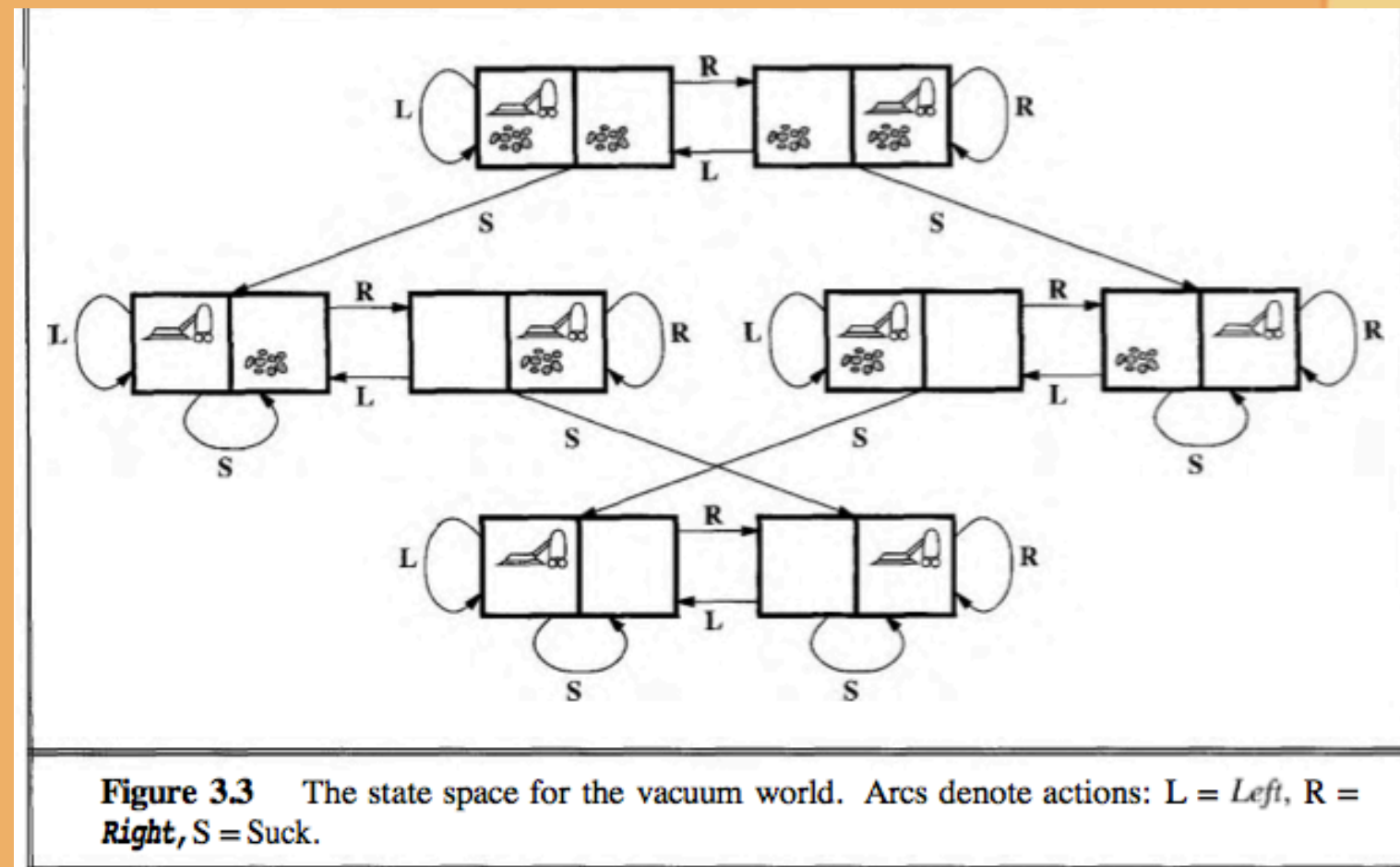
- States?
- Actions?
- Goal test?
- Path cost?



Vacuum world state space graph



- States?
- integer dirt and robot location
- Actions? Left, Right, Suck
- Goal test?
- No dirt at all locations
- Path cost?
- 1 per action



The 8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- States?
- Actions?
- Goal test?
- Path cost?

The 8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- States? locations of tiles
- Actions? move blank left, right, up, down
- Goal test? a given goal state
- Path cost? 1 per move
- Optimal solution of n -Puzzle family is NP-hard!!!



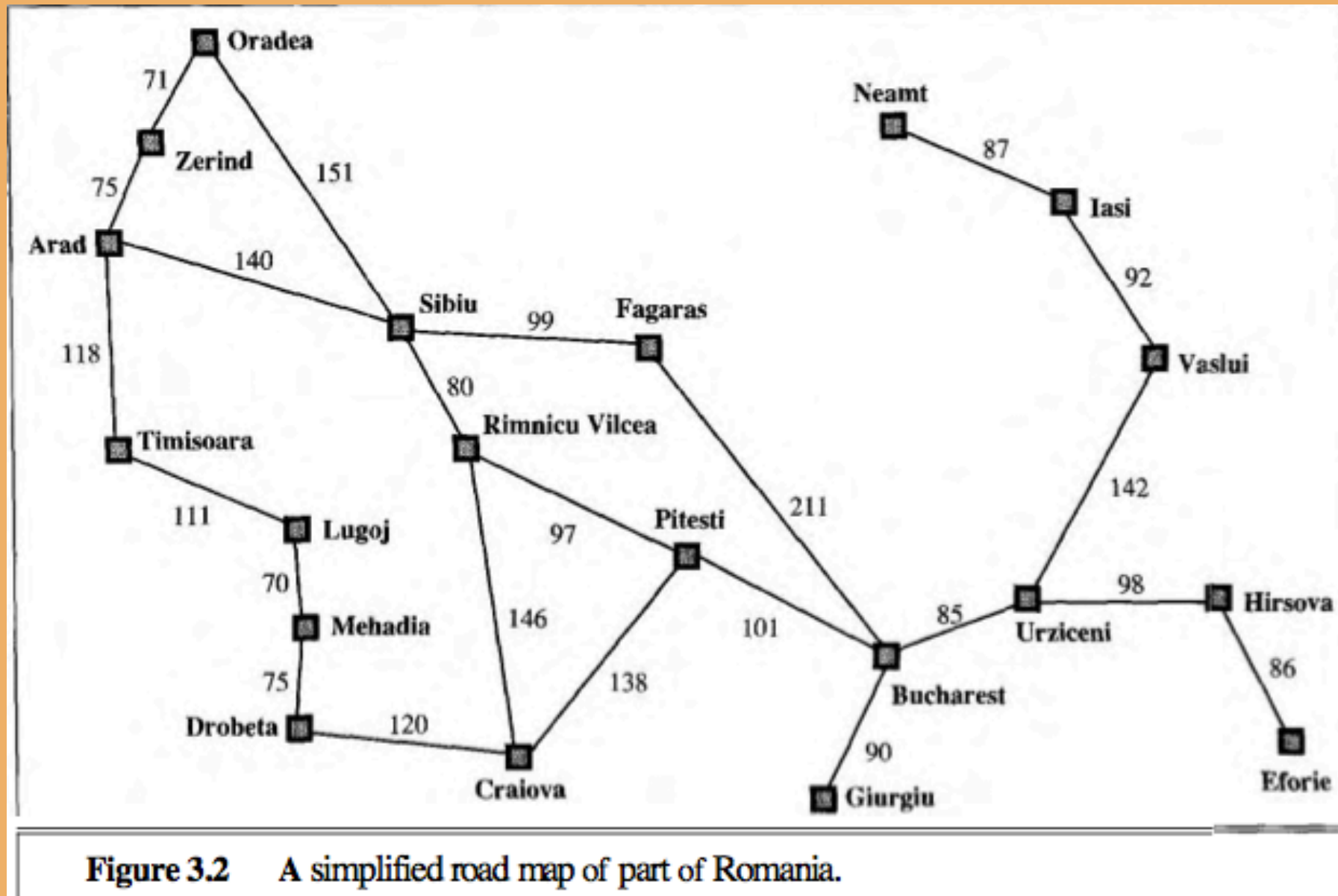
Tree search algorithms

- After formulating the problem, we have to solve it. It can be done by a search through the state space.
- Basic idea:

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  ioop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
```



Example



General tree-search algorithm

function TREE-SEARCH(*problem*, *fringe*) **returns** a solution, or failure

fringe $\leftarrow$ INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *fringe*)

loop do

if EMPTY?(*fringe*) **then return** failure

node $\leftarrow$ REMOVE-FIRST(*fringe*)

if GOAL-TEST[*problem*] applied to STATE[*node*] succeeds
 then return SOLUTION(*node*)

fringe $\leftarrow$ INSERT-ALL(EXPAND(*node*, *problem*), *fringe*)

function EXPAND(*node*, *problem*) **returns** a set of nodes

successors $\leftarrow$ the empty set

for each (*action*, *result*) **in** SUCCESSOR-FN[*problem*](STATE[*node*]) **do**

s $\leftarrow$ a new NODE

 STATE[*s*] $\leftarrow$ *result*

 PARENT-NODE[*s*] $\leftarrow$ *node*

 ACTION[*s*] $\leftarrow$ *action*

 PATH-COST[*s*] $\leftarrow$ PATH-COST[*node*] + STEP-COST(STATE[*node*], *action*, *result*)

 DEPTH[*s*] $\leftarrow$ DEPTH[*node*] + 1

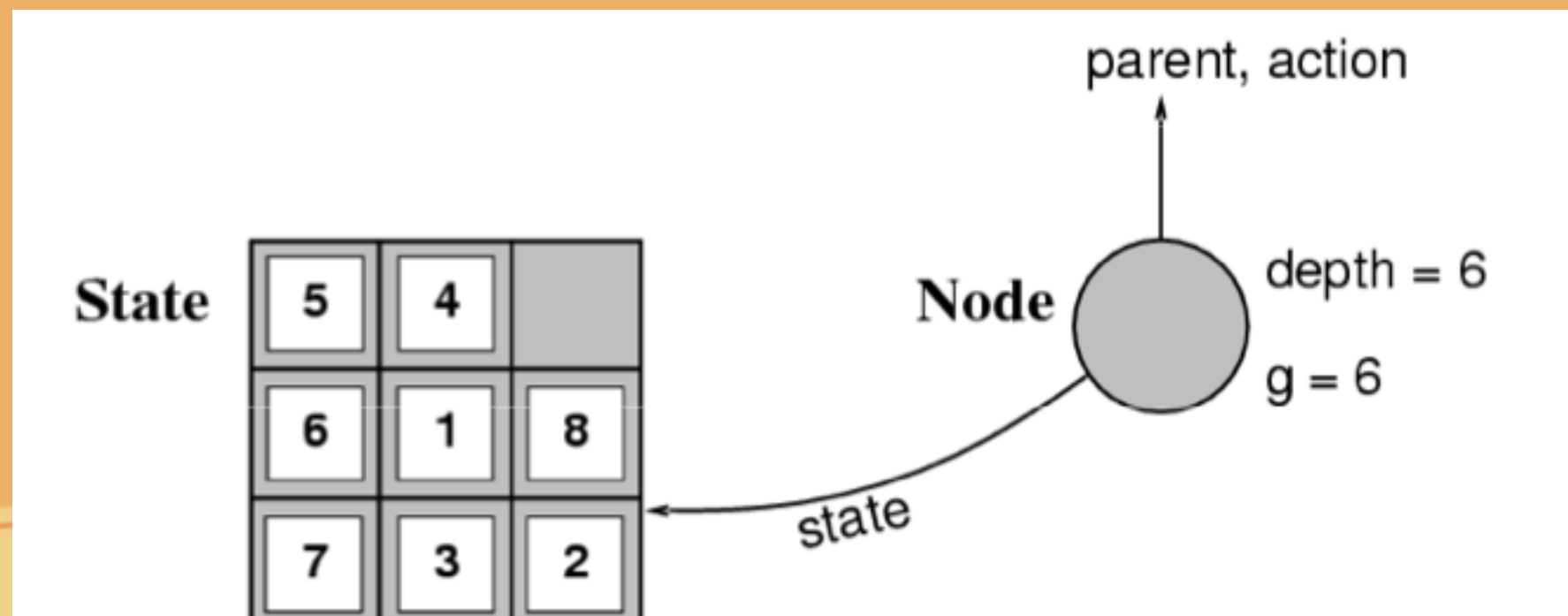
 add *s* to *successors*

return *successors*

Figure 3.9 The general tree-search algorithm. (Note that the *fringe* argument must be an empty queue, and the type of the queue will affect the order of the search.) The *SOLUTION* function returns the sequence of actions obtained by following parent pointers back to the root.

Implementation: States vs Nodes

- *A state is a representation of a physical configuration*
- *A node is a data structure constituting part of a search tree, including: state, parent node, action, path-cost and depth (the number of steps along the path from the initial state).*



Implementation: States vs Nodes

- The “Expand” function creates new nodes, filling in the various fields and using the “SuccessorFn” of the problem to create the corresponding states.
- A fringe is a collection of nodes that have been generated but not expanded.

Search strategies



- *A search strategy is defined by picking the order of node expansion.*
- *Strategies are evaluated along the following dimensions:*
 - *completeness: does it always find a solution if one exists?*
 - *time complexity: number of nodes generated*
 - *space complexity: How much memory is needed to perform the search*
 - *optimality: does it always find a least-cost solution?*



Search strategies

- *Time and space complexity are measured in terms of*
 - *b : maximum branching factor of the search tree*
 - *d : depth of the least-cost solution*
 - *m : maximum depth of the state space*

Uninformed search strategies

- Also called “blind search”.
- No additional information about states beyond that provided in the problem definition.
- All they can do is generate successors and distinguish a goal state from any non-goal state.
- Different from the informed strategies or heuristic strategies.



Uninformed search strategies

- *Several uninformed search strategies:*
 - *Breadth-first search.*
 - *Uniform-cost search.*
 - *Depth-first search.*
 - *Depth-limited search.*
 - *Iterative deepening search.*





Breadth-first search

- *Expand shallowest unexpanded node.*
- *Implementation:*
 - *fringe is a First-In-First-Out queue, i.e., new successors go at end.*





Breadth-first search

- Complete?

- Time?

- Space?

- Optimal?

→ HOMEWORK!





Uniform-cost search

- Expand least-cost unexpanded node.
 - Implementation:
 - fringe = queue ordered by path cost.
 - Equivalent to breadth-first if step costs all equal.
 - Complete?
 - Time? Space? Optimal?
→ HOMEWORK!
- 



Depth-first search

- *Expand deepest unexpanded node.*
- *Implementation:*
 - *fringe = Last-In-First-Out, i.e., put successors at front*
- *Complete?*
- *Time?*
- *Space?*
- *Optimal?*
 - *HOME WORK!!!*



Depth-limited search

- A depth-first search with depth limit.

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff  
  return RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)
```

```
function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff  
  cutoff_occurred?  $\leftarrow$  false  
  if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)  
  else if DEPTH[node] = limit then return cutoff  
  else for each successor in EXPAND(node, problem) do  
    result  $\leftarrow$  RECURSIVE-DLS(successor, problem, limit)  
    if result = cutoff then cutoff_occurred?  $\leftarrow$  true  
    else if result  $\neq$  failure then return result  
  if cutoff_occurred? then return cutoff else return failure
```

Iterative deepening search

- Sometimes used in deep-first search, that finds the best depth limit.

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure  
  inputs: problem, a problem  
  
  for depth  $\leftarrow 0$  to  $\infty$  do  
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)  
    if result  $\neq$  cutoff then return result
```

Figure 3.14 The iterative deepening search algorithm, which repeatedly applies depth-limited search with increasing limits. It terminates when a solution is found or if the depth-limited search returns *failure*, meaning that no solution exists.

Avoid repeated states

- Failure to detect repeated states can turn a linear problem into an exponential one!

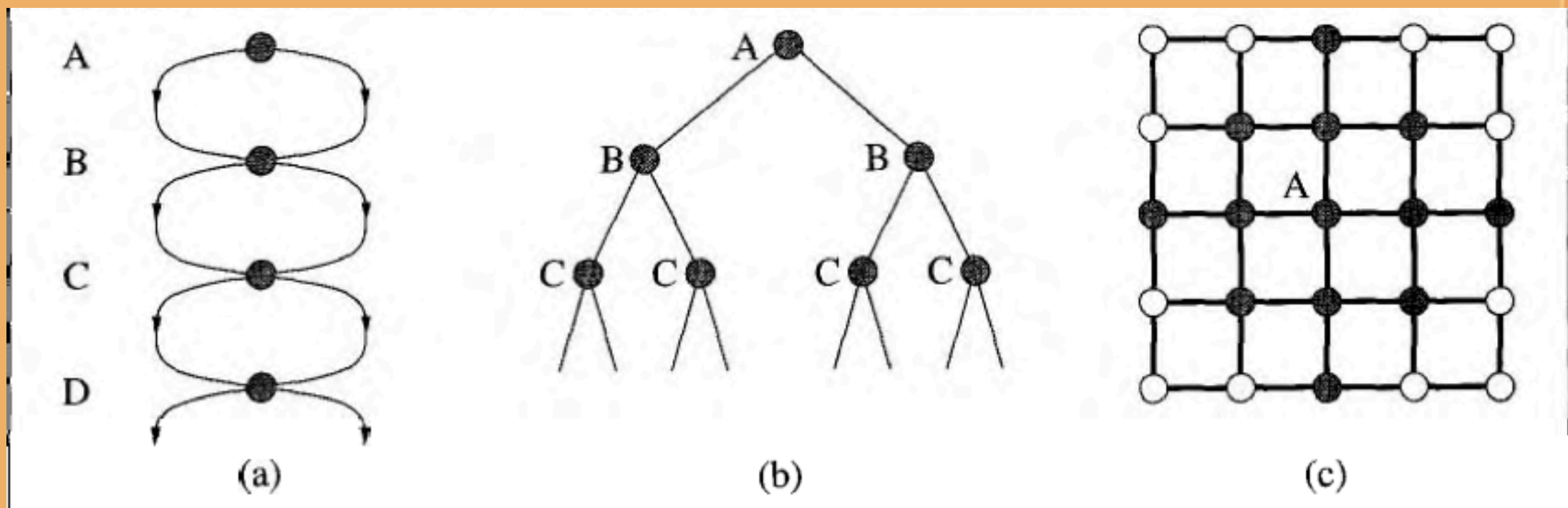


Figure 3.18 State spaces that generate an exponentially larger search tree. (a) A state space in which there are two possible actions leading from A to B, two from B to C, and so on. The state space contains $d + 1$ states, where d is the maximum depth. (b) The corresponding search tree, which has 2^d branches corresponding to the 2^d paths through the space. (c) A rectangular grid space. States within 2 steps of the initial state (A) are shown in gray.

Avoid repeated states

```
function GRAPH-SEARCH(problem, fringe) returns a solution, or failure  
closed  $\leftarrow$  an empty set  
fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)  
loop do  
  if EMPTY?( fringe) then return failure  
  node  $\leftarrow$  REMOVE-FIRST(fringe)  
  if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)  
  if STATE[node] is not in closed then  
    add STATE[node] to closed  
    fringe  $\leftarrow$  INSERT-ALL(EXPAND(node, problem), fringe)
```

Figure 3.19 The general graph-search algorithm. The set *closed* can be implemented with a hash table to allow efficient checking for repeated states. This algorithm assumes that the first path to a state *s* is the cheapest (see text).



Bidirectional search

- *Simultaneously search both forward (from the initial state) and backward (from the goal state)*
- *Stop when the two searches meet.*

